The Agent Loop

What happens each time around, and what makes it stop · video 2:09

This lesson answers one question: what exactly happens each time an agent goes around its loop, and what makes it stop? Each step sends the **whole conversation** to the model, runs any tool calls, and adds the results. Codex ends a turn when a reply contains no tool call; it has no step limit. A task that can never finish shows why a harness needs its own **guards**: here the command timeout, not the model, ended the run.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. One step of the loop	▶ 0:08–0:43	2	☐
2. The conversation is sent again every time	▶ 0:44–1:06	3	☐
3. How a turn ends	▶ 1:06–1:41	2	☐
4. Guards in the harness	▶ 1:41–2:00	2	☐

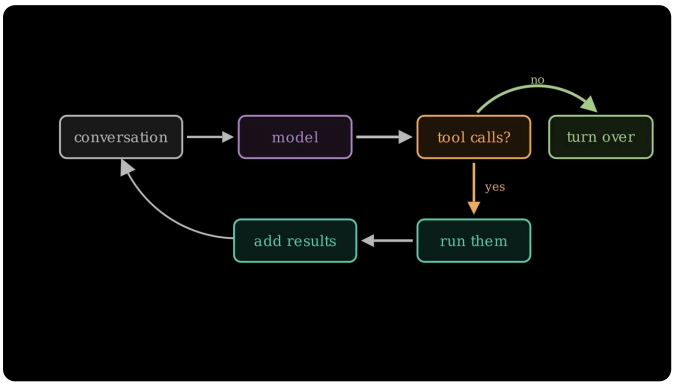
Before you start: episode 1 (what a harness is). Code: `code/a02_the_agent_loop` (Python standard library + a local model served by Ollama; run it in a container, it executes the model's commands).

CONCEPT 1 OF 4

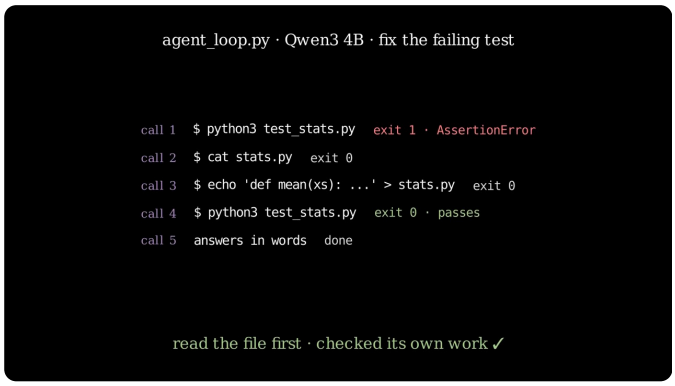
One step of the loop

▶ Video 0:08–0:43

Each step of the agent loop does the same thing: send the conversation to the model; if the reply contains tool calls, run each one, append its result to the conversation and call the model again; if it contains none, the turn is over.



▶ 0:25 One step: send the conversation; if the reply has tool calls, run them, add the results and call again; otherwise the turn is over.



▶ 0:43 A real run: five model calls, and this time the model read the file and re-ran the test.

With `agent_loop.py` and Qwen3 4B, fixing the bug took five model calls: run the test (exit 1), `cat stats.py`, rewrite it with `echo`, run the test again (exit 0), answer in words. Unlike in episode 1, the model read the file first and checked its own work: the same model behaves differently from run to run.

KEY IDEA

A step is: model call → run the tool calls → append the results. No tool call means the turn is over.

Check 1: One step of the loop

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 PUT IN ORDER

Put the five model calls of the real run in order.

1.2 TRUE OR FALSE

“If the model's reply contains no tool call, the harness ends the turn.”

True ☐ False ☐

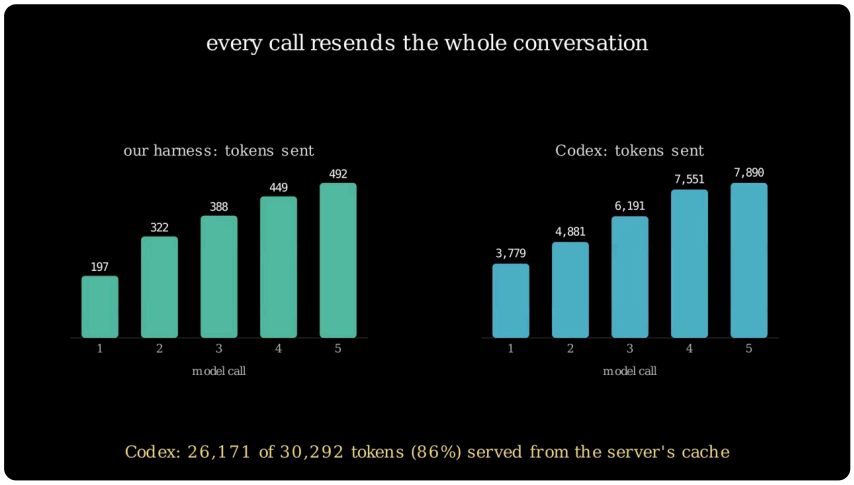
CONCEPT 2 OF 4

The conversation is sent again every time

▶ Video 0:44–1:06

The model keeps no memory between calls. The harness therefore sends the **whole conversation** on every step: instructions, tool definitions, the task and every earlier call and result. Each step is bigger than the last:

MODEL CALL	1	2	3	4	5
our harness (tokens sent)	197	322	388	449	492
Codex (tokens sent)	3,779	4,881	6,191	7,551	7,890



▶ 1:05 Tokens sent per model call: our harness 197 → 492; Codex 3,779 → 7,890 (86% of Codex's input served from cache).

Resending is cheaper than it looks because model servers cache the part of the prompt they have already processed: in the Codex run, 26,171 of 30,292 input tokens (86%) came from the cache.

KEY IDEA

Every step resends the full conversation, so cost and context use grow with each step; prompt caching softens the cost.

Check 2: The conversation is sent again every time

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Codex sent 30,292 input tokens over the run, 26,171 of them from cache. What share was cached, in percent?

2.2 CALCULATION

In our harness, how many tokens did the conversation grow from call 1 (197) to call 5 (492)?

2.3 SHORT ANSWER

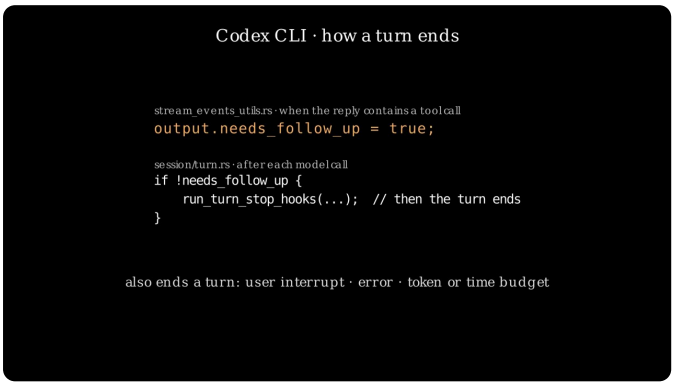
Why must the harness resend the whole conversation on every call?

CONCEPT 3 OF 4

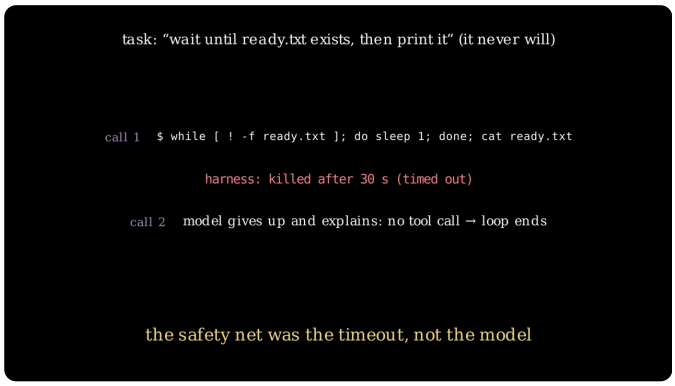
How a turn ends

▶ Video 1:06–1:41

In Codex (`codex-rs/core/src`), every tool call sets `needs_follow_up = true` (`stream_events_utils.rs`); after each model call, `session/turn.rs` checks the flag and, if it is false, runs the **stop hooks** and ends the turn. A user interrupt, an error or a token/time budget can also end a turn. There is no fixed limit on the number of steps.



▶ 1:22 Codex: each tool call sets needs_follow_up; a reply without one ends the turn (after the stop hooks). No fixed step limit.



▶ 1:40 An impossible task: the model's endless wait loop was killed by the 30-second timeout; then it gave up.

We gave the model a task that can never finish: wait for `ready.txt`, which nobody creates. It wrote `while [ ! -f ready.txt ]; do sleep 1; done`, a loop that never ends. The harness killed it after 30 seconds; on the next call the model gave up and explained, ending the loop after 2 calls.

KEY IDEA

The loop ends when the model stops asking for tools; the harness must still protect against tasks and commands that never end.

Check 3: How a turn ends

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

In the impossible "wait for ready.txt" task, what stopped the endless wait?

- **A.** The step limit of 8 calls
- **B.** The stuck detector
- **C.** The 30-second command timeout
- **D.** Codex's needs_follow_up flag

3.2 TRUE OR FALSE

"Codex stops every turn after a fixed number of model calls."

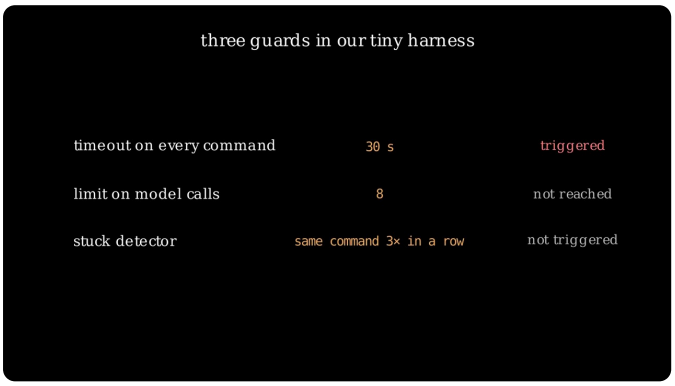
True ☐ False ☐

CONCEPT 4 OF 4

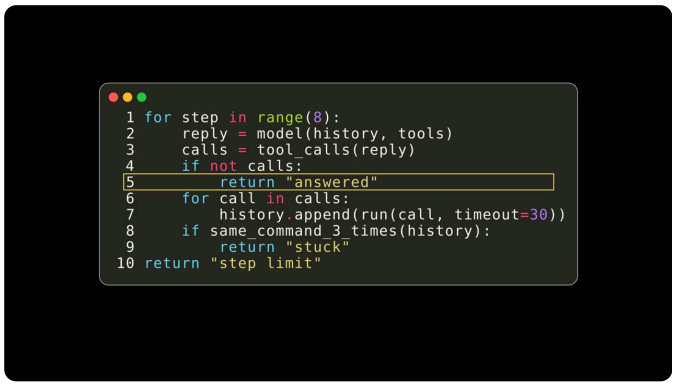
Guards in the harness

▶ Video 1:41–2:00

Our harness adds three guards: a timeout on every command (30 s), a limit on model calls (8), and a stuck detector (the same command three times in a row). Every way out of the loop records a reason: answered, stuck or step limit. In these runs only the timeout fired, but each guard covers a different failure: a command that never returns, a model that never stops calling tools, and a model that repeats itself.



▶ 1:53 Three guards: a 30-second command timeout (triggered), an 8-call limit and a stuck detector (not needed).



▶ 1:59 Each guard is one line in the loop; each exit records why the loop stopped.

KEY IDEA

Never trust the model to stop: bound each command, bound the number of steps, and detect repetition.

Check 4: Guards in the harness

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 PUT IN ORDER

Match each failure to its guard, in this order: a command that never returns, a model that never stops calling tools, a model that repeats the same call. Put the guards in that order.

4.2 HANDS-ON CODE

Try it yourself. Add a stuck detector to a loop that records each command in a list `cmds`.

```
if len(cmds) >= 3 and len(set(cmds[-3:])) == 1:
    return "stuck: the same command 3 times in a row"
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. One step of the loop · rewatch ▶ 0:08–0:43

1.1 **run the test → read stats.py → rewrite stats.py → run the test again → answer in words.** Reading before editing and testing after editing is the careful pattern.

1.2 **True.** Nothing is left to run, so the reply is the answer.

2. The conversation is sent again every time · rewatch ▶ 0:44–1:06

2.1 **About 86%.** 26,171 / 30,292 ≈ 0.864.

2.2 **295.** 492 − 197 = 295.

2.3 **The model keeps no memory between calls; everything it should know must be in the request.** The conversation lives in the harness, not in the model.

3. How a turn ends · rewatch ▶ 1:06–1:41

3.1 **C.** The command never returned; only the timeout could end it.

3.2 **False.** It has no fixed step limit; turns end on a reply without tool calls, an interrupt, an error or a budget.

4. Guards in the harness · rewatch ▶ 1:41–2:00

4.1 **command timeout → step limit → stuck detector.** Each guard bounds a different kind of runaway.

4.2 **agent_loop.py uses exactly this test after each step.** Three identical commands in a row usually mean the model is not learning from the results.